NanoVDB `ReadAccessor` Benchmark — OLD vs NEW, Acc=\<0,1,2> vs Acc=\<0>, CPU vs GPU

Benchmarks comparing the `NANOVDB_USE_OLD_ACCESSOR` caching behaviour (ON vs OFF) and `ReadAccessor<0,1,2>` (full 3-level cache, default) vs `ReadAccessor<0>` (leaf-only cache) across access patterns, on CPU (single- and multi-threaded) and GPU. Results are collected from five machines: a **Razer laptop**, a **laptop**, a **desktop**, a **workstation**, and a **MacBook Air M3**. The OLD/NEW comparison for the Laptop and Desktop used only `ReadAccessor<0,1,2>`; the full accessor-type comparison (both `<0,1,2>` and `<0>`) was run on the Workstation, the Razer Laptop, and the MacBook Air M3.

> **Methodology correction (MacBook Air M3 and Workstation sections).** The Laptop / Desktop / Razer Laptop sections sampled a cube inside a **fog-volume sphere**. That interior is stored as **active tiles at upper internal nodes**, so every access resolved at level 2 and *never reached a leaf* — which makes the leaf-only `ReadAccessor<0>` look artificially bad and does not measure genuine leaf traversal. The MacBook Air M3 and Workstation sections use a **corrected** benchmark whose coordinates are harvested from the grid's **active leaf voxels** (a narrow-band level set sphere); 100 % of accesses resolve at a leaf node. Consequently the **absolute ns/access in those sections are ~10× larger** than the pre-correction sections — that difference is the methodology (real multi-level leaf traversal vs. re-hitting a small set of cached upper-node tiles), **not** the hardware. Compare OLD-vs-NEW and `<0,1,2>`-vs-`<0>` only *within* a section, never across the correction boundary.

What is being measured

The `ReadAccessor` caches the tree nodes visited on the previous access so a spatially nearby access can skip the traversal from the root.

- **OLD** (`NANOVDB_USE_OLD_ACCESSOR` defined): the `get()` method uses `if constexpr` + `else` chaining. For `getValue` (operation LEVEL=0), the `if constexpr(LEVEL<=0)` branch is taken and the `else` discards all higher-level cache checks — so the 3-level accessor falls straight to a **root traversal** on any leaf-cache miss, effectively behaving like a leaf-only accessor.
- **NEW** (`NANOVDB_NO_OLD_ACCESSOR` defined): the `else` is removed, so all three cache-level checks execute unconditionally. A leaf-cache miss can still hit the **level-1 / level-2** cache instead of restarting at the root.
- **`ReadAccessor<0,1,2>`** (`DefaultReadAccessor`): maintains leaf, lower-internal, and upper-internal node caches. With NEW, the full 3-level check is active. With OLD, the extra levels are stored but never checked on a read — making it functionally equivalent to a larger `ReadAccessor<0>` struct.
- **`ReadAccessor<0>`**: caches only the leaf node. On a leaf miss it always goes straight to root. Unaffected by the OLD/NEW flag (no level-1/2 to check).

Methodology

What the numbers measure

Every reported figure is the **wall-clock time of a single random-access read** into the grid, i.e. one `accessor.getValue(ijk)` call, expressed in **nanoseconds per lookup** (`ns/lookup`). It is a pure *read-path* micro-benchmark: no values are written, no math is done on the result beyond a running sum (kept `volatile` / written to an output array so the compiler cannot elide the work).

- **CPU-1T** — one thread walks the coordinate list serially with a single accessor. This is a **latency** number: the cost of one dependent lookup.
- **CPU-MT** — the same total work split across all hardware threads (`nanovdb::util::forEach` → `tbb::parallel_for`), each grain using its own accessor. This is a **throughput** number: total time ÷ total lookups with all cores busy.
- **GPU** — the coordinate list is uploaded to the device; each thread processes a 32-coord chunk with its own accessor. Timed with CUDA events (kernel time only, excluding the one-time grid upload). Also a **throughput** number.

The only thing that differs between the OLD and NEW columns is the accessor's cache-fallback logic (a compile-time switch); the data, coordinates, and harness are identical.

The test data

A **fog-volume sphere** built with `nanovdb::tools::createFogVolumeSphere<float>(radius=500, voxelSize=1.0, halfWidth=3.0)`. Its measured properties:

Property	Value
Value type	`float`
Index bounding box	`[-500,-500,-500] … [500,500,500]` (1001^3 voxels)
Dense voxel count of bbox	1,003,003,001 (~1.00 B)
Active voxels	**523,592,077 (~523.6 M)**
Fill ratio (active / dense bbox)	**52.2 %**
Upper internal nodes (level 2, 32^3)	8
Lower internal nodes (level 1, 16^3)	272
Leaf nodes (level 0, 8^3)	82,608
Grid size in memory	188,484,640 bytes (~180 MB)

So it is a **fully populated solid ball** — the interior is all active voxels (the 52 % fill ratio is just the volume of a sphere inside its cube, $\pi/6 \approx 0.524$). It is *sparse* in the VDB sense (only the ball is stored, not the empty corners), but the region the benchmark actually samples is **100 % active**: all access patterns are confined to a cube well inside radius 500, so every lookup lands on a real leaf and traverses the tree for real — we are **not** measuring empty-space shortcuts, we are measuring genuine node traversal and cache reuse.

How much work, how it is averaged

Point patterns (Sequential / LeafJump / NodeJump / Random)	1,048,576 lookups each

Stencil	262,144 centres × 27 neighbours = 7,077,888 lookups
Sampled region	cube of half-extent 256 (points) / `[0,64)³` dense cube (stencil), both fully inside the ball
Repetition	each configuration run **7 times; the median is reported** (robust to OS jitter / turbo ramp)
Warm-up	GPU does one untimed warm-up launch before the 7 timed trials

Hardware / build

Four machines were benchmarked; results for each are in separate sections below.

	Razer Laptop	Laptop	Desktop	Workstation
CPU	Intel Core i7-9750H — 6 cores / 12 threads; TBB `parallel_for`, 4096-coord grains	32 HW threads; TBB `parallel_for`, 4096-coord grains	AMD Ryzen 9 9950X — 16 cores / 32 threads; same TBB settings	AMD Ryzen Threadripper PRO 7975WX — 32 cores / 64 threads; same TBB settings
GPU	NVIDIA Quadro RTX 5000 with Max-Q Design (SM 7.5); 32-coord chunks/thread, 128 threads/block	NVIDIA RTX 5000 Ada Generation Laptop GPU (SM 8.9); 32-coord chunks/thread, 128 threads/block	NVIDIA RTX PRO 6000 Blackwell Max-Q Workstation Edition (SM 12.0); same CUDA settings	NVIDIA RTX 6000 Ada Generation (SM 8.9, 49 GB); same CUDA settings
Build	Release (`-O3 -DNDEBUG`), C++17 / CUDA 17	same	same	same
OLD vs NEW	selected at compile time per target (`-DNANOVDB_USE_OLD_ACCESSOR` vs `-DNANOVDB_NO_OLD_ACCESSOR`)	same	same	same

Access patterns

Pattern	Stride / shape	Cache behaviour
Sequential	1	leaf (level-0) cache always hot
LeafJump	8 (= leaf dim)	leaf cache cold; level-1 warm (NEW only)
NodeJump	128 (= lower-node dim)	leaf + level-1 cold; level-2 warm (NEW only)
Random	uniform in cube	all caches cold — both fall to root
Stencil	3×3×3 = 27 neighbours per centre, dense sweep	mostly same-leaf hits; boundary neighbours spill to level-1 (NEW rescues)

Results — MacBook Air M3 (Apple M3, 8-core CPU; corrected benchmark, CPU only)

This is the only section using the corrected leaf-sampling methodology (see the note above). Coordinates are harvested from the active leaf voxels of a narrow-band level set sphere: `nanovdb::tools::createLevelSetSphere<float>(radius=256, voxelSize=1.0, halfWidth=3.0)`.

Property	Value
Grid	narrow-band level set sphere (radius 256, voxel 1.0, halfWidth 3.0)
Active leaf voxels harvested	4,939,794
Leaf nodes (level 0)	26,600
Lower internal nodes (level 1)	80
Upper internal nodes (level 2)	8
Access resolving at leaf (verified)	**100 %** (was 0 % pre-correction)
CPU	Apple M3, 8 HW threads; TBB `parallel_for`, 4096-coord grains
GPU	none (no CUDA on Apple silicon) — CPU only
Build	Release (`-O3 -DNDEBUG`), C++17; each printed figure is the median of 7 trials

CPU single-threaded — ns per access (latency)

Pattern	OLD \<0,1,2>	OLD \<0>	NEW \<0,1,2>	NEW \<0>	OLD→NEW \<0,1,2>
Sequential	20.98	20.35	**19.79**	20.04	1.06×
LeafJump	135.47	108.52	**53.17**	104.53	**2.55×**
NodeJump	123.07	109.29	**90.77**	107.62	1.36×
Random	218.39	176.14	185.11	**165.49**	1.18×
Stencil (ns/lookup)	52.06	47.97	**40.44**	47.43	1.29×

CPU 8-threaded — ns per access (throughput)

Pattern	OLD \<0,1,2>	OLD \<0>	NEW \<0,1,2>	NEW \<0>	OLD→NEW \<0,1,2>
Sequential	3.98	3.76	**3.82**	3.73	1.04×
LeafJump	26.13	21.02	**11.63**	20.70	**2.25×**
NodeJump	26.83	20.96	**18.36**	20.69	1.46×
Random	34.67	30.94	37.92	**28.90**	0.91×
Stencil (ns/lookup)	11.51	11.36	**8.79**	10.05	1.31×

Stencil detail — ns per whole 27-neighbour stencil

Mode	OLD \<0,1,2>	OLD \<0>	NEW \<0,1,2>	NEW \<0>
CPU 1 thread	1405.57	1295.28	**1091.88**	1280.73

Mode	OLD \<0,1,2>	OLD \<0>	NEW \<0,1,2>	NEW \<0>
CPU 8 threads	310.72	306.74	**237.44**	271.33

Takeaways — MacBook Air M3 (corrected benchmark)

- `ReadAccessor<0>` **is unchanged by OLD→NEW** (within ~1–6 % on every pattern and thread count, i.e. noise). This is the clean control the corrected sampling makes possible — the OLD/NEW flag only touches the multi-level cache chaining in `<0,1,2>`.
- `<0,1,2>` **NEW wins exactly where multi-level caching applies**: **LeafJump 2.55× (1T) / 2.25× (8T)** — the headline — plus NodeJump 1.36–1.46× and Stencil 1.29–1.31×. Sequential is a tie (access stays within one leaf, which both variants cache).
- **Random is the one place** `<0,1,2>` **does not help** (0.91× on 8T; the 1.18× on 1T is inflated because random picks among the clustered narrow-band voxels retain some upper-node locality). For pure scatter, `<0>` remains the better choice — it avoids the extra cache-level checks.

Results — Desktop (AMD Ryzen 9 9950X / RTX PRO 6000 Blackwell, SM 12.0) — pre-correction, to be regenerated

> These numbers predate the leaf-sampling fix and measure upper-node tile access, not leaf traversal (see the note in the intro). To be regenerated with the corrected benchmark.

Combined — ns per lookup (one `getValue` call), lower = faster

The stencil row is the per-lookup cost inside a 27-neighbour sweep.

Pattern	CPU-1T OLD	CPU-1T NEW	CPU-MT OLD	CPU-MT NEW	GPU OLD	GPU NEW
Sequential (stride 1)	2.45	**1.44**	0.26	**0.18**	0.035	**0.020**
LeafJump (stride 8)	2.06	**1.41**	0.21	**0.16**	0.035	**0.020**
NodeJump (stride 128)	1.85	**1.64**	0.22	**0.21**	0.028	**0.024**
Random (uniform)	**7.19**	9.93	**0.47**	0.57	**0.039**	0.047
Stencil (3×3×3, 27-pt)	2.143	**1.371**	0.170	**0.102**	0.0587	**0.0275**

Legend: CPU-1T = single thread · CPU-MT = 32 threads (TBB) · GPU = RTX PRO 6000 Blackwell.

OLD → NEW speedup (>1 = the fix is faster)

Pattern	CPU-1T	CPU-MT	GPU
Sequential	1.70×	1.44×	1.75×
LeafJump	1.46×	1.31×	1.75×
NodeJump	1.13×	1.05×	1.17×
Random	0.72×	0.82×	0.83×

Pattern	CPU-1T	CPU-MT	GPU
Stencil	**1.56×**	**1.67×**	**2.14×**

Detailed tables — Desktop

CPU, single-threaded (latency) — ns per lookup

Pattern	OLD	NEW	OLD→NEW
Sequential	2.45	**1.44**	1.70×
LeafJump	2.06	**1.41**	1.46×
NodeJump	1.85	**1.64**	1.13×
Random	**7.19**	9.93	0.72×
Stencil	2.143	**1.371**	1.56×

CPU, 32 threads (throughput) — ns per lookup

Pattern	OLD	NEW	OLD→NEW	MT speedup (NEW)
Sequential	0.26	**0.18**	1.44×	8.0×
LeafJump	0.21	**0.16**	1.31×	8.8×
NodeJump	0.22	**0.21**	1.05×	7.8×
Random	**0.47**	0.57	0.82×	17.4×
Stencil	0.170	**0.102**	1.67×	13.4×

GPU (throughput) — ns per lookup

Pattern	OLD	NEW	OLD→NEW
Sequential	0.035	**0.020**	1.75×
LeafJump	0.035	**0.020**	1.75×
NodeJump	0.028	**0.024**	1.17×
Random	**0.039**	0.047	0.83×
Stencil	0.0587	**0.0275**	2.14×

Stencil (3×3×3) — reported per whole 27-neighbour stencil

Platform / Mode	OLD ns/stencil	NEW ns/stencil	OLD→NEW
CPU 1 thread	57.86	**37.01**	1.56×
CPU 32 threads	4.58	**2.74**	1.67×
GPU	1.5862	**0.7413**	2.14×

Fair platform comparison — NEW accessor, full hardware (ns per lookup)

Workload	CPU-1T	CPU-MT	GPU	GPU vs CPU-MT
Sequential	1.44	0.18	0.020	9.0×
Random	9.93	0.57	0.047	12.1×
Stencil	1.371	0.102	0.0275	3.7×

Results — Laptop (RTX 5000 Ada Generation Laptop GPU, SM 8.9) — pre-correction, to be regenerated

> These numbers predate the leaf-sampling fix and measure upper-node tile access, not leaf traversal (see the note in the intro). To be regenerated with the corrected benchmark.

Combined — ns per lookup (one `getValue` call), lower = faster

The stencil row is the per-lookup cost inside a 27-neighbour sweep.

Pattern	CPU-1T OLD	CPU-1T NEW	CPU-32T OLD	CPU-32T NEW	GPU OLD	GPU NEW
Sequential (stride 1)	3.57	**1.96**	0.56	**0.46**	0.058	0.058
LeafJump (stride 8)	3.82	**2.21**	0.55	**0.49**	0.057	0.057
NodeJump (stride 128)	3.94	**3.05**	0.58	**0.53**	0.054	0.055
Random (uniform)	**11.47**	15.32	**0.82**	1.04	**0.088**	0.095
Stencil (3×3×3, 27-pt)	4.61	**2.13**	0.322	**0.191**	0.086	**0.040**

Legend: CPU-1T = single thread · CPU-32T = 32 threads (TBB) · GPU = RTX 5000 Ada.

OLD → NEW speedup (>1 = the fix is faster)

Pattern	CPU-1T	CPU-32T	GPU
Sequential	1.82×	1.22×	1.00×
LeafJump	1.73×	1.12×	1.00×
NodeJump	1.29×	1.09×	0.98×
Random	0.76×	0.79×	0.93×
Stencil	**2.17×**	**1.68×**	**2.17×**

Detailed tables — Laptop

CPU, single-threaded (latency) — ns per lookup

Pattern	OLD	NEW	OLD→NEW
Sequential	3.57	**1.96**	1.82×
LeafJump	3.82	**2.21**	1.73×
NodeJump	3.94	**3.05**	1.29×
Random	**11.47**	15.32	0.76×
Stencil	4.61	**2.13**	2.17×

CPU, 32 threads (throughput) — ns per lookup

Pattern	OLD	NEW	OLD→NEW	MT speedup (NEW)
Sequential	0.56	**0.46**	1.22×	4.3×
LeafJump	0.55	**0.49**	1.12×	4.5×

Pattern	OLD	NEW	OLD→NEW	MT speedup (NEW)
NodeJump	0.58	**0.53**	1.09×	5.7×
Random	**0.82**	1.04	0.79×	14.7×
Stencil	0.322	**0.191**	1.68×	11.1×

GPU (throughput) — ns per lookup

Pattern	OLD	NEW	OLD→NEW
Sequential	0.058	**0.058**	1.00×
LeafJump	0.057	**0.057**	1.00×
NodeJump	0.054	0.055	0.98×
Random	**0.088**	0.095	0.93×
Stencil	0.086	**0.040**	2.17×

Stencil (3×3×3) — reported per whole 27-neighbour stencil

Platform / Mode	OLD ns/stencil	NEW ns/stencil	OLD→NEW
CPU 1 thread	124.48	**57.49**	2.17×
CPU 32 threads	8.70	**5.17**	1.68×
GPU	2.32	**1.07**	2.17×

Fair platform comparison — NEW accessor, full hardware (ns per lookup)

Workload	CPU-1T	CPU-32T	GPU	GPU vs CPU-32T
Sequential	1.96	0.46	0.058	7.9×
Random	15.32	1.04	0.095	10.9×
Stencil	2.13	0.191	0.040	4.8×

Results — Workstation (AMD Ryzen Threadripper PRO 7975WX / RTX 6000 Ada, SM 8.9) — corrected benchmark

This section uses the **corrected leaf-sampling benchmark** (same methodology as the MacBook Air M3 section). Coordinates are drawn from the active leaf voxels of a narrow-band level set sphere (`createLevelSetSphere<float>(radius=256, voxelSize=1.0, halfWidth=3.0)`); every lookup resolves at a leaf node.

Property	Value
Grid	narrow-band level set sphere (radius 256, voxel 1.0, halfWidth 3.0)
Active leaf voxels harvested	4,939,794
Leaf nodes (level 0)	26,600
Lower internal nodes (level 1)	80
Access resolving at leaf	**100 %**
CPU	AMD Ryzen Threadripper PRO 7975WX — 32 cores / 64 threads; TBB `parallel_for`, 4096-coord grains
GPU	

Property	Value
	NVIDIA RTX 6000 Ada Generation (SM 8.9, 49 GB); 32-coord chunks/thread, 128 threads/block
Access count per pattern	1,048,576 per pattern; stencil centres up to 262,144 (filtered for full 27-tap activity)
Repetition	7 trials, median reported

CPU single-threaded — ns per access (latency)

Pattern	OLD \<0,1,2>	OLD \<0>	NEW \<0,1,2>	NEW \<0>
Sequential	1.59	1.29	**1.26**	**1.10**
LeafJump	8.06	6.44	**4.41**	6.52
NodeJump	5.30	**4.32**	4.67	4.47
Random	23.09	**19.04**	27.28	**19.00**
Stencil (ns/lookup)	1.857	1.561	**1.761**	1.716

CPU 64-threaded — ns per access (throughput)

Pattern	OLD \<0,1,2>	OLD \<0>	NEW \<0,1,2>	NEW \<0>
Sequential	0.22	**0.18**	0.22	**0.18**
LeafJump	0.36	0.37	**0.31**	0.32
NodeJump	0.29	**0.27**	0.34	0.29
Random	0.74	**0.68**	0.81	**0.67**
Stencil (ns/lookup)	0.066	0.077	**0.065**	0.073

GPU — ns per access (throughput)

Pattern	OLD \<0,1,2>	OLD \<0>	NEW \<0,1,2>	NEW \<0>
Sequential	0.027	**0.026**	**0.026**	**0.026**
LeafJump	0.063	0.063	**0.045**	0.063
NodeJump	0.086	0.086	**0.058**	0.083
Random	**0.133**	**0.132**	0.147	**0.132**
Stencil (ns/lookup)	**0.0587**	**0.0586**	0.0669	**0.0583**

OLD → NEW speedup by accessor type

Pattern	CPU-1T \<0,1,2>	CPU-1T \<0>	CPU-MT \<0,1,2>	CPU-MT \<0>	GPU \<0,1,2>	GPU \<0>
Sequential	1.26×	**1.17×**	1.00×	1.00×	1.04×	1.00×
LeafJump	**1.83×**	0.99×	**1.16×**	1.16×	**1.40×**	1.00×
NodeJump	1.13×	0.97×	0.85×	0.93×	**1.48×**	1.04×
Random	0.85×	1.00×	0.91×	1.01×	0.90×	1.00×
Stencil	**1.05×**	0.91×	**1.02×**	1.05×	0.88×	1.01×

Key: **`ReadAccessor<0>` is essentially unchanged by OLD→NEW** — confirming that the fix only affects the 3-level accessor. NEW `<0,1,2>` wins strongly for LeafJump (CPU 1T 1.83×, GPU 1.40×) and NodeJump on GPU (1.48×). Stencil shows only a small CPU win and a slight GPU regression — explained below.

Stencil detail — ns per whole 27-neighbour stencil

Platform	OLD \<0,1,2>	OLD \<0>	NEW \<0,1,2>	NEW \<0>
CPU 1 thread	50.14	42.14	**47.53**	46.34
CPU 64 threads	1.79	2.08	**1.75**	1.97
GPU	**1.5859**	**1.5828**	1.8057	**1.5742**

The GPU stencil **regresses 14 % with NEW `<0,1,2>`** (1.8057 vs 1.5859 ns/stencil). With the corrected methodology the stencil centres are active interior-band voxels whose full 27-neighbour neighbourhood is also active — meaning all taps stay within the narrow leaf band, the level-0 (leaf) cache is already hot, and the extra level-1/2 checks in NEW add overhead with no benefit. This contrasts with the fog-sphere stencil (old methodology) where sampling a dense cube caused cross-leaf spills that the level-1 cache caught.

Best-accessor recommendation (NEW mode, this machine)

Pattern	Best choice	Why
Sequential	`<0>` (1.10 ns 1T)	Leaf cache stays hot; level-1/2 checks add marginal cost
LeafJump	`<0,1,2>` (4.41 ns)	Level-1 cache rescues leaf misses; 1.83× vs `<0>`
NodeJump	`<0,1,2>` (4.67 ns) or `<0>` (4.47 ns)	Level-2 gives slight edge on GPU; effectively tied on CPU
Random	`<0>` (19.00 ns)	All levels miss; extra checks are pure cost
Stencil (CPU)	`<0,1,2>` (1.761 ns/lkp 1T)	Small edge; taps stay in-leaf so benefit is modest
Stencil (GPU)	`<0>` (0.0583 ns/lkp)	NEW `<0,1,2>` regresses; leaf cache already hot

Fair platform comparison — best accessor (NEW), full hardware

Workload	CPU-1T	CPU-64T	GPU	GPU vs CPU-64T
Sequential (`<0>`)	1.10	0.18	0.026	6.9×
LeafJump (`<0,1,2>`)	4.41	0.31	0.045	6.9×
Random (`<0>`)	19.00	0.67	0.132	5.1×
Stencil CPU (`<0,1,2>`)	1.761	0.065	—	—
Stencil GPU (`<0>`)	—	—	0.0583	~1.1× vs CPU-64T

Results — Razer Laptop (Intel Core i7-9750H / Quadro RTX 5000 Max-Q, SM 7.5)

Both `ReadAccessor<0,1,2>` and `ReadAccessor<0>` were benchmarked under both OLD and NEW compile-time modes. The CPU is a Turing-era mobile part with 6 physical cores / 12 HW threads; the GPU is an NVIDIA Quadro RTX 5000 with Max-Q Design (Turing, SM 7.5).

CPU single-threaded — ns per lookup (latency)

Pattern	OLD \<0,1,2>	OLD \<0>	NEW \<0,1,2>	NEW \<0>
Sequential	5.17	5.44	**3.21**	5.26

Pattern	OLD \<0,1,2>	OLD \<0>	NEW \<0,1,2>	NEW \<0>
LeafJump	5.38	5.61	**3.67**	5.51
NodeJump	5.31	5.32	**4.72**	5.32
Random	**11.42**	**11.94**	16.13	**11.53**
Stencil (ns/lookup)	5.200	5.700	**2.959**	5.654

CPU 12-threaded — ns per lookup (throughput)

Pattern	OLD \<0,1,2>	OLD \<0>	NEW \<0,1,2>	NEW \<0>
Sequential	1.50	1.36	**0.68**	1.22
LeafJump	1.25	1.46	**0.72**	1.30
NodeJump	1.47	1.80	**1.11**	1.24
Random	1.70	**1.66**	2.42	1.81
Stencil (ns/lookup)	1.553	1.462	**0.738**	1.314

GPU — ns per lookup (throughput)

Pattern	OLD \<0,1,2>	OLD \<0>	NEW \<0,1,2>	NEW \<0>
Sequential	0.302	0.309	0.344	**0.283**
LeafJump	0.293	0.302	0.336	**0.277**
NodeJump	0.261	0.268	0.275	**0.248**
Random	0.363	0.378	0.388	**0.345**
Stencil (ns/lookup)	0.1999	0.2098	**0.0849**	0.1858

OLD → NEW speedup by accessor type

Pattern	CPU-1T \<0,1,2>	CPU-1T \<0>	CPU-MT \<0,1,2>	CPU-MT \<0>	GPU \<0,1,2>	GPU \<0>
Sequential	**1.61×**	1.03×	**2.21×**	1.11×	0.88×	1.09×
LeafJump	**1.47×**	1.02×	**1.74×**	1.12×	0.87×	1.09×
NodeJump	**1.12×**	1.00×	**1.32×**	1.45×	0.95×	1.08×
Random	0.71×	1.04×	0.70×	0.92×	0.94×	1.10×
Stencil	**1.76×**	1.01×	**2.10×**	1.11×	**2.35×**	1.13×

Key: `ReadAccessor<0>` **is essentially unchanged by OLD→NEW on CPU**, in line with all other machines. On the GPU (SM 7.5 Turing), `<0>` shows a small but consistent ~1.09–1.13× improvement — an outlier not seen on newer architectures, possibly noise or a compiler code-gen difference on this older target.

Stencil detail — ns per whole 27-neighbour stencil

Platform	OLD \<0,1,2>	OLD \<0>	NEW \<0,1,2>	NEW \<0>
CPU 1 thread	140.39	153.90	**79.89**	152.65
CPU 12 threads	41.94	39.48	**19.92**	35.48
GPU	5.3960	5.6641	**2.2926**	5.0156

Best-accessor recommendation (NEW mode, this machine)

Pattern	Best choice	Why
Sequential	`<0,1,2>` (3.21 ns)	Level-1 cache stays warm between leaf-boundary crossings
LeafJump	`<0,1,2>` (3.67 ns)	Same — level-1 rescues leaf misses
NodeJump	`<0,1,2>` (4.72 ns)	Level-2 cache pays off; slightly faster than `<0>` (5.32 ns)
Random	`<0>` (11.53 ns)	All levels miss; extra checks are pure cost
Stencil	`<0,1,2>` (2.959 ns/lkp)	Boundary neighbours caught by level-1; ~1.91× over `<0>`

Unlike the Workstation (where NodeJump NEW `<0,1,2>` regressed 2× vs OLD), here the level-2 cache benefit slightly outweighs the extra check overhead, so `<0,1,2>` remains the right choice for NodeJump as well.

Fair platform comparison — best accessor (NEW), full hardware

Workload	CPU-1T	CPU-12T	GPU	GPU vs CPU-12T
Sequential (`<0,1,2>`)	3.21	0.68	0.283	2.4×
Random (`<0>`)	11.53	1.81	0.345	5.2×
Stencil (`<0,1,2>`)	2.959	0.738	0.0849	8.7×

Cross-machine comparison (Razer Laptop vs Laptop vs Desktop vs Workstation)

Scope: Laptop and Desktop data uses `ReadAccessor<0,1,2>` only and the pre-correction fog-sphere methodology. The Workstation (this run) and Razer Laptop use both accessor types. The Workstation section is also on the corrected leaf-sampling methodology; absolute ns/access values are not directly comparable to the pre-correction sections. Compare OLD→NEW ratios and `<0,1,2>`-vs-`<0>` ratios within each section.

1. LeafJump is the biggest win with corrected methodology (Workstation)

With coordinates drawn from actual active leaf voxels, LeafJump (one voxel per leaf in storage order) exercises a realistic cache pattern: the level-0 cache always misses, but adjacent leaves share the same lower-internal node — exactly the case NEW `<0,1,2>` rescues.

Platform	LeafJump CPU-1T OLD→NEW `<0,1,2>`	LeafJump GPU OLD→NEW `<0,1,2>`
Workstation (corrected, RTX 6000 Ada)	**1.83×**	**1.40×**
Razer Laptop (pre-correction, RTX 5000 Turing)	1.47×	0.87×

The corrected workstation result (1.83×, 1.40×) likely represents the true benefit more accurately than the pre-correction figures. The Razer's GPU regression was already flagged as a SM 7.5 anomaly.

2. GPU NodeJump wins with corrected methodology

Platform	NodeJump GPU OLD→NEW `<0,1,2>`
Workstation (corrected)	**1.48×**
Razer Laptop (pre-correction)	0.95×
Laptop (pre-correction)	0.98×

With real level-2 cache locality (one voxel per lower-internal node from the actual grid topology, staying within the same upper node), the level-2 cache hit path in NEW is exercised and pays off on GPU. This was not visible in the synthetic NodeJump (fixed stride=128) used in pre-correction runs.

3. Stencil GPU: corrected methodology reverses the finding

Machine	GPU Stencil OLD→NEW `<0,1,2>`	Methodology
Workstation (corrected)	**0.88× (regression)**	active-voxel, 27-tap-active filter
Razer Laptop (pre-correction)	2.35×	fog sphere, dense cube
Laptop (pre-correction)	2.17×	fog sphere, dense cube
Desktop (pre-correction)	2.14×	fog sphere, dense cube

With the corrected stencil (centres where all 27 neighbours are active leaf voxels), taps stay within the narrow band and the level-0 leaf cache is already hot. The NEW accessor's extra level-1/2 checks add overhead with no cache-hit benefit → 14 % regression. The pre-correction ~2× win was real for the fog-sphere use case (where cross-leaf spills from the dense cubic interior made the level-1 catch meaningful), but does not represent narrow-band workloads.

4. CPU MT regression unchanged: tied to 64-thread count and `<0,1,2>`

Pattern	Workstation (corrected) CPU-MT `<0,1,2>`
Sequential	1.00×
LeafJump	**1.16×**
NodeJump	0.85×
Random	0.91×
Stencil	**1.02×**

With the corrected methodology the MT picture is similar: small gains for LeafJump and near neutrality everywhere else. The large pre-correction MT stencil win (1.57×) does not appear in the corrected run, consistent with the stencil GPU finding above.

5. `ReadAccessor<0>` unaffected by OLD→NEW on all platforms

Confirmed: `<0>` OLD→NEW ratios remain 0.97–1.05× across all patterns, platforms, and methodologies. The fix is entirely in the 3-level `<0,1,2>` accessor.

6. Random always regresses for \<0,1,2>; \<0> is neutral

For `<0,1,2>`: all caches miss → 0.85× on CPU-1T, 0.90× on GPU (corrected run). For `<0>`: no extra checks → 1.00× on all platforms. Hardware-independent.

Summary and bottom line

The NEW fix is unambiguously beneficial for `ReadAccessor<0,1,2>` on stencil and coherent point patterns. The gains are 1.5–2.3× and portable across CPU and GPU. `ReadAccessor<0>` (leaf-only) sees no benefit from the fix and no regression — it is effectively unchanged.

For production use with the NEW accessor, the choice of accessor type matters:

Use case	Recommended accessor
Stencil / convolution / dense sweep	`ReadAccessor<0,1,2>` (NEW gains 2×)
Sequential / LeafJump access	`ReadAccessor<0,1,2>` (NEW gains 1.4–2.3×)
NodeJump / random scatter	`ReadAccessor<0>` (avoids regression, same or faster)
Unknown / mixed	`ReadAccessor<0,1,2>` — wins on the common workloads

Per-machine takeaways

- **LeafJump is the primary win** with the corrected methodology (active-voxel patterns): **1.83× on CPU-1T and 1.40× on GPU** for NEW `<0,1,2>` on the Workstation. This is the pattern that directly exercises the level-1 cache rescue: leaf cache misses but the lower internal node is still cached.
- **Stencil results are methodology-dependent.** Pre-correction (fog sphere, dense cube): `<0,1,2>` gains ~2× on GPU across all machines. Corrected (level set sphere, filtered active-voxel centres): the GPU stencil regresses 14 % with NEW `<0,1,2>` because all 27 neighbours already land in the leaf band, the leaf cache stays hot, and the extra checks add cost. `ReadAccessor<0>` is neutral in both methodologies.
- **GPU NodeJump now wins with corrected patterns (1.48× on Workstation GPU)**: with one representative voxel per actual lower-internal node, the level-2 cache hit path in NEW pays off. This was not captured by the synthetic stride=128 pattern used in pre-correction runs.
- **Random always regresses for `<0,1,2>`** (0.85–0.90× across all platforms): all cache levels miss, extra checks are pure overhead. `<0>` is neutral (1.00×). Hardware-independent.
- **`ReadAccessor<0>` is unaffected by OLD→NEW** on all platforms, patterns, and thread counts. Its code path (leaf check → root) has no level-1/2 checks to change.
- **CPU multi-thread scaling on 64-thread Threadripper PRO**: LeafJump achieves 13–16× (MT speedup vs 1T), Random 29–31×, Sequential 7×. The stencil MT gain (1.02×) is minimal under the corrected methodology, consistent with the stencil's reduced cross-leaf spill rate.

Building & running

```
# Configure with the benchmark (and CUDA, for the GPU variant) enabled
cmake -S . -B build -DNANOVDB_BUILD_BENCHMARK=ON -DNANOVDB_USE_CUDA=ON

# Build all four executables
cmake --build build --target \
```

```
  bench_accessor_old bench_accessor_new \
  bench_accessor_cuda_old bench_accessor_cuda_new -j

# Run — each binary prints results for both ReadAccessor<0,1,2> and ReadAccessor<0>
cd build/nanovdb/nanovdb/benchmark
./bench_accessor_old        # CPU, OLD accessor, both <0,1,2> and <0>
./bench_accessor_new        # CPU, NEW accessor, both <0,1,2> and <0>
./bench_accessor_cuda_old   # GPU, OLD accessor, both <0,1,2> and <0>
./bench_accessor_cuda_new   # GPU, NEW accessor, both <0,1,2> and <0>
```

The OLD vs NEW behaviour is selected at compile time per target (`-DNANOVDB_USE_OLD_ACCESSOR` vs `-DNANOVDB_NO_OLD_ACCESSOR`). Both accessor types (`ReadAccessor<0,1,2>` and `ReadAccessor<0>`) are run within each binary.

Files

File	Purpose
`BenchPatterns.h`	Shared access-pattern generation (identical coords for CPU & GPU)
`BenchAccessor.cc`	CPU benchmark (single- and multi-threaded)
`BenchAccessorCuda.cu`	GPU (CUDA) benchmark
`CMakeLists.txt`	Builds the OLD/NEW × CPU/GPU targets